

CMR Specialist Automation Agent

STATUS: Master Execution Plan VERSION: 1.0 TARGET DATE: 31 August 2026 REGION: Africa/Lagos

FOUNDATIONAL ARCHITECTURAL LAW

The AI is replaceable. The case record, knowledge history and audit trail are the product memory. A conversation is temporary. A case is durable. An answer is useful only when its source and effect can be traced. AI reasons; software decides.

1. Architectural Principles & Boundaries

- **1. Case Record First:** Each individual can own multiple cases. Status belongs strictly to a case, never to the person. Every transition is an immutable event.
- **2. Knowledge is Published:** Documents are versioned, reviewed, and atomically activated. Retrieval matches active versions only; older answers retain original citations.
- **3. AI Reasons; Software Decides:** Models classify, extract facts, and draft responses. Authorization, state transitions, document publication, and audit records remain deterministic code.
- **4. Trace Every Action:** Store who acted, what changed, retrieved chunks, model parameters, token counts, latency, and error identifiers in durable storage.
- **5. Stateless Compute Scales:** FastAPI API containers hold no local durable state. PostgreSQL + pgvector remains the single source of truth.
- **6. Split Only Under Pressure:** Start as a modular monolith with one worker service. Defer microservices and Kafka until measured bottlenecks demand them.

SYSTEM BOUNDARY CONSTRAINT

The product explains official procedures and manages support workflows. It **MUST NEVER** fabricate payment confirmations, ownership status, NIMC availability, or authoritative facts without verified integration or human sign-off.

2. Unified Module Directory Layout

All phases build additive, cohesive modules into a single unified directory tree:

Repository Structure (Modular Monolith)

```
app/
  api/           # Customer, Support Console, and Operations Dashboard routes
  cases/         # Durable case domain, models, named commands, strict state machine
  knowledge/     # Document versioning, PDF parsing, chunking, atomic publication
  ai/           # Binary intent routing, vector retrieval (pgvector), inference & trace
  platform/     # Async DB engine, JWT/RBAC auth, PII-masked audit, telemetry
worker/         # Background tasks: PDF ingestion, embeddings, notifications, exports
migrations/     # Versioned Alembic migrations
tests/          # Unit, integration, grounding retrieval, and load test suites
```

3. Phased Execution Roadmap

Phase 0: Foundation & Platform Infrastructure

Establish repository layout, container environments, Pydantic typed configuration, async PostgreSQL session factory, telemetry middleware with X-Request-ID, append-only audit logging with PII masking, and role-based access control (RBAC).

- **[app/config.py]** Typed settings for DB, Redis, S3, LLM keys, and token limits.
- **[app/platform/database.py]** Async SQLAlchemy engine, async session maker, declarative Base with UUID primary keys.
- **[app/platform/auth.py]** JWT generation/validation, password hashing (bcrypt), RBAC dependencies (customer, support, admin, auditor).
- **[app/platform/audit.py]** Append-only audit sink with automatic PII masking (BVN, NIN, passwords, card numbers).
- **[app/platform/telemetry.py]** FastAPI middleware injecting X-Request-ID and structured structlog context.

PHASE 0 EXIT CHECK

API and worker containers start cleanly via docker-compose; GET /health returns 200 OK with Request ID.

Phase 1: Durable Core & Case State Machine

Implement core database entities (Person, Case, CaseEvent, Conversation, Message, ExtractedFact) and enforce strict case lifecycle transitions via named commands.

- **[app/cases/models.py]** Person, Case, CaseEvent, Conversation, Message, ExtractedFact models.
- **[app/cases/state_machine.py]** Strict transitions: NEW -> TRIAGED -> WAITING <-> IN_REVIEW -> RESOLVED -> CLOSED.
- **[app/cases/commands.py]** Named transition commands: Triage, Assign, RequestInfo, CustomerReply, Resolve, Reopen, Close.
- **[migrations/versions/0001_initial_schema.py]** Alembic migration creating all core tables and pgvector extension.

PHASE 1 EXIT CHECK

Case survives database restart with an intact chronological timeline. Illegal transitions raise HTTP 400/422.

Phase 2: Customer Surface

Customer-facing endpoints for asking informational questions, grounded answers, opening cases without repeating facts, and tracking status.

- **POST /api/v1/customer/questions:** Ingest question, execute binary routing, return grounded answer with citations or prompt case creation.
- **POST /api/v1/customer/cases:** Open support case with extracted facts pre-populated from chat history.
- **GET /api/v1/customer/cases/{id}:** Retrieve case timeline, messages, requested actions, and attachments.
- **POST /api/v1/customer/cases/{id}/messages:** Submit customer response (automatically shifts case from WAITING to IN_REVIEW).

PHASE 2 EXIT CHECK

A customer can ask questions, open a case, reply, and track case progress end-to-end.

Phase 3: Support Console Surface

Internal staff console allowing agents and supervisors to search people, view cross-case histories, filter queues, assign work, execute transitions via named commands, and record internal notes.

- **GET /api/v1/support/people** Cross-case search by name, phone, email, or verified identity identifier.
- **GET /api/v1/support/cases** Queue filtering by status, priority (LOW, NORMAL, HIGH, URGENT), category, assignee, and SLA.
- **POST /api/v1/support/cases/{id}/commands/{cmd}** Execute named commands (Triage, Assign, RequestInfo, Resolve, Reopen, Close).
- **POST /api/v1/support/cases/{id}/notes** Record internal staff notes and decisions.

PHASE 3 EXIT CHECK

Support staff can triage, assign, and resolve cases completely without requiring direct SQL access.

Phase 4: Knowledge Base & Vector Publishing

Document management pipeline supporting PDF parsing, text extraction, semantic chunking, pgvector embedding generation, and atomic version activation.

- **app/knowledge/models.py**: Document, DocumentVersion (DRAFT, ACTIVE, RETIRED), Chunk (with Vector(1536)).
- **worker/tasks/ingestion.py**: Async background task extracting PDF text, generating chunks, and computing vector embeddings.
- **app/knowledge/service.py**: Atomic version switch: marks active as RETIRED and draft as ACTIVE in a single transaction.
- **Publishing Guarantee**: A version is either entirely active or inactive. Customers never see partial ingestion chunks.

PHASE 4 EXIT CHECK

A new PDF can be uploaded in draft, processed in background, published atomically, cited, and rolled back safely.

Phase 5: AI Grounding, Routing & Traceability

Binary intent routing, active-version vector retrieval, prompt synthesis enforcing strict citations, and full AI execution trace recording.

- **app/ai/router.py** Binary Router: classifies query into INFORMATIONAL (knowledge query) vs SUPPORT_REQUEST (open/update case).
- **app/ai/retrieval.py** Cosine similarity search querying ONLY chunks linked to ACTIVE DocumentVersions.
- **app/ai/inference.py** Grounded answer generator enforcing [Doc: title, Chunk: id] citations and boundary guardrails.
- **app/ai/models.py (AIRun)** Persists model name, prompt template version, retrieved chunks, token counts, latency, and feedback.

PHASE 5 EXIT CHECK

Every AI answer can be completely reconstructed and verified from stored AIRun records and citations.

Phase 6: Operations Dashboard & Reliability

Observability dashboard providing health probes, operational metrics across 4 key pillars, searchable PII-masked audit logs, and incident diagnosis runbooks.

PILLAR	TRACKED METRICS	OPERATIONAL TARGET
Service Health	Request volume, error rate, p50/p95 latency, container uptime	Monthly availability >= 99.9%
Customer Flow	Answers delivered, cases opened, case age, resolution time	Non-AI p95 latency < 500 ms
AI Quality	Retrieval hit rate, fallback rate, token cost, user feedback	Completed AI answer p95 < 12 sec
Knowledge	Active versions, failed ingestions, publish age, zero-result queries	Doc ingestion to publish < 5 min

PHASE 6 EXIT CHECK

An operator can identify a failing request by its Request ID and view the exact error trace and database events.

Phase 7: Launch, Verification & Testing

Execute end-to-end integration tests, seed realistic procedural knowledge and test cases, perform load testing at 2x peak traffic, and verify disaster recovery runbooks.

- **Database Seeder (scripts/seed_db.py):** Populates default admin/support accounts, active procedural documents, and sample cases.
- **E2E Integration Suite (tests/integration/):** Verifies full lifecycle: Question -> Case Created -> Triaged -> In Review -> Resolved -> Closed.
- **Disaster Recovery Drill:** Point-in-time PostgreSQL restore and atomic container rollback verified.

PHASE 7 EXIT CHECK

Automated test suite passes 100%; service meets all SLO targets under simulated 2x peak load.

4. Component Dependency & Flow Matrix

MODULE	CORE ROLE	UPSTREAM TRIGGER	OUTPUT STATE
app/platform	Database, Auth, Audit, Telemetry	All HTTP requests & tasks	Postgres, Redis, Structured Logs
app/cases	Case lifecycle & state machine	Customer & Support commands	CaseEvent, Case, ExtractedFact
app/knowledge	Ingestion & atomic publishing	Support document upload/publish	DocumentVersion, Chunks
app/ai	Intent routing, retrieval & LLM	Customer question endpoint	AIRun, Grounded Answer, Citations
worker/tasks	Async document & vector worker	Redis task queue	Vector embeddings, Notifications
app/api	Three distinct surface routers	End users / Consoles / Dashboard	Dispatches to Domain Services